MSA 8770 – Milestone 3: Insurance Claims Intake Triage

Human Review, Safe Event Handling, and Production Operation

Team 7: Raveena Pavuluri, Nikhithareddy Obili, Ishita Varma Kalindindi, Kiarash Rezaei, Tejaswini Waghmare

Code, sample inputs, setup and test instructions: github.com/raveenapavuluri03/Team7_ClaimsTriage

LLM: gpt-oss:20b running **locally** through the Ollama desktop app (no claim text leaves the machine) · November 2026

Summary. Milestone 3 turns the simulated adjuster decision into a real checkpoint: every payout waits in a saved review, is paid only after an authorized approval, and is paid at most once even when events repeat, the worker crashes, or a payment confirmation is lost. We ran 13 new checks and the 19 Milestone 2 scenarios on the real model; all 32 passed (Section 4). Human review, not the model, is the main production cost (Section 5).

1. Updated Architecture and Decisions

Milestone 2 ended with a working S1–S10 pipeline whose human steps were simulated and whose payment was a log line. We changed the parts that must be real for Milestone 3 and left the S1–S10 logic as it was. Rows marked **FB** answer the Milestone 2 feedback.

Area	Milestone 2	Milestone 3	Why
Adjuster decision (H2)	Function call that set APPROVED/DENIED	Saved review in SQLite with approval scope, 48 h expiry, escalation (review.py)	Brief requires one real checkpoint. **FB**
External effects	Log lines	Simulated payment and notification services that record rows; idempotency keys	Effects must be verifiable in the receiver
State	JSON file per claim	JSON (atomic write) plus SQLite for reviews, events, payments	Count real effects; detect repeated events
Fallback risk score	Score 0 shown as LOW	Band UNVERIFIED; approval needs explicit acknowledgement	A failed model must not look safe. **FB**
resume_claim()	No status check; counters kept; corrections skipped S4	Status guard; counters reset (history kept); corrections re-validated	Duplicate and late events. **FB**
Model hosting	Not stated	Stated: local Ollama; no data leaves the machine	Decides cost and deployment. **FB**

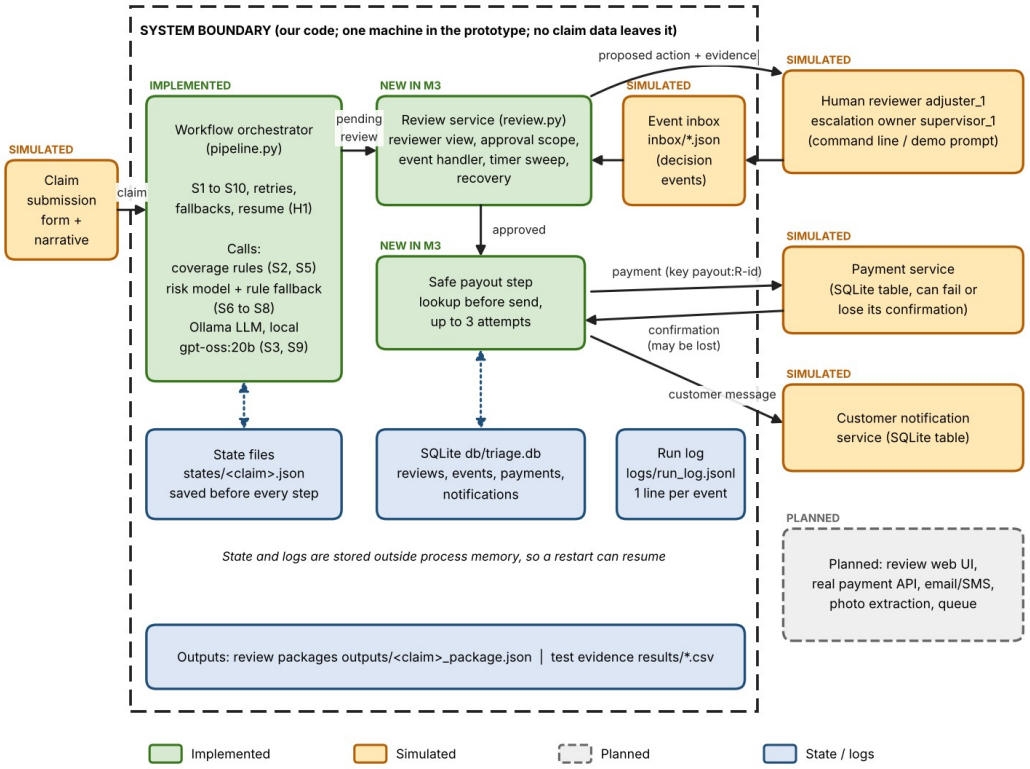


Figure 1: Component architecture. The dashed line is the system boundary; green = implemented, amber = simulated, grey = planned, blue = state and logs.

Design Decision Records (D1–D3 new in Milestone 3; D4–D6 updated)

Decision	Rationale	Alternative	Trade-off	Evidence
D1 Approval before every payout, saved as a pending review with 48 h + 24 h deadlines	Money leaves the company and is hard to recall	Auto-pay low-risk claims, audit later	Reviewer time per claim; slower small claims	Tests 1–3: payment only after APPROVE; 0 payments after reject or expiry
D2 Idempotency keys (event_id, payout:review_id) and ask the receiver before any retry	A repeat or retry must not pay twice; a lost confirmation makes blind retry unsafe	Never retry, or retry blindly	One extra lookup per payment; needs a receiver with keys or lookup	Tests 4–7: 7 payments recorded for 7 paid claims across duplicate, crash, lost-confirmation and outage
D3 SQLite plus atomic state files now, PostgreSQL in production	Durable, queryable, no server to run	JSON only; queue + workflow engine	One writer machine; no multi-worker scaling yet	State survives a simulated crash (test 5); compare-and-set lets one decision win (not load tested)
D4 (updated) Mandatory human gate: H2 now real	Wrongful payout or denial is costly	Full automation	Higher reviewer workload	M2: 0 claims paid without H2 by convention; M3: enforced by code (test 1). Reviewer time and override rate still unmeasured
D5 (updated) ML fraud score with rule fallback, fallback labelled UNVERIFIED	ML recall 0.735 vs 0.673 for rules; fallback keeps the run going	Show fallback as LOW; block approval	Extra acknowledgement click	Test 8: same claim scored 0 by fallback vs 0.165 by ML; approval without acknowledgement refused
D6 (updated) Local Ollama gpt-oss:20b	No claim data leaves the machine	Hosted API	Needs a 16 GB machine; 33 s per claim	634 input and 553 output tokens per claim (Section 5)

2. Human Oversight Design and Implemented Checkpoint

Oversight by Action

Action	Effect	Oversight	Justification (stakes, reversibility)
Pay a claim	Spends money	**Approval before execution**	High stakes; a payment is hard to recall
Notify customer of denial	Sends a message, changes the record	**Approval before execution** (the reviewer's reject is the approval)	Affects the customer; reversible only by appeal
Ask customer for missing information	Sends a message	**Monitoring with intervention** (clerk can edit or stop)	Low stakes; easy to correct
Write review package, state and logs	Changes internal records	**Later audit** (run log, daily reconciliation)	Internal only; fully reversible
Risk score and explanation	Informs a person	**No separate review**: a human reads it before any action	Advisory; cannot act by itself

The Implemented Checkpoint (H2: payout review)

- **Who reviews:** adjuster_1 (assigned). **Escalation owner:** supervisor_1, who takes over after 48 hours without a decision. The checkpoint applies to every payout, because payout is irreversible; exception cases (missing data, mismatch, unclear coverage) still go to H1 first.
- **What the reviewer sees** (real output for claim TC1; the fallback case adds a warning line):

```
REVIEW R-336e18  status: PENDING  claim: TC1   Expires: 2026-11-02T12:42:24
Assigned to: adjuster_1    Escalation owner: supervisor_1
PROPOSED ACTION : PAYOUT of $50,590.00 to policy 429027 (Auto Basic)
CONTENT VERSION : a2ade8c58f67
EVIDENCE   Multi-vehicle collision 2015-02-27, police report YES; claimed $51,590.00
           (limit $100,000, deductible $1,000); risk 0.165 LOW
(ml_logistic_regression)
RULE       Coverage COVERED: payout = claim - deductible (Auto Basic)
CONFLICTS  none     [fallback case: ! RISK SCORE IS A FALLBACK ... band UNVERIFIED]
IF YOU APPROVE this authorizes ONLY: PAYOUT | claim TC1 | $50,590.00 | version
  a2ade8c58f67 | valid until 2026-11-02T12:42:24 | one payment (key payout:R-336e18)
IF YOU REJECT: no payment, DENIED.  IF NOBODY DECIDES: escalated, then withheld.
```

Approval authorizes	Detail
Action and target	One PAYOUT to the policy of one claim; no other claim, amount or action
Content version	Hash of the extracted fields, form amount, coverage result and payout. If anything changes (for example a reviewer correction), a new review replaces the old one and the old approval is refused
Expiry	48 h for the adjuster, 24 h more for the supervisor; after that nothing can be approved
Count	Exactly one payment (idempotency key payout:review_id)

Outcome	What happens	End state
Approval	State saved as APPROVED_EXECUTING, then the payment service is asked (lookup first); customer notified once	PAID
Rejection	No payment; customer notified once; reason kept in the review	DENIED
Timeout	After 48 h escalate to supervisor_1; after 24 h more the payout is withheld and the case needs a manual decision. Later approvals are ignored	ESCALATED_TO_SUPERVISOR, then WITHHELD_EXPIRED

3. Event Handling and Saved State

Event path. A reviewer decision is a JSON event (event_id, review_id, decision, reviewer) written to `inbox/` (a simulated webhook) and processed by a worker (`review_cli.py worker`). A timer event

(`sweep`) escalates or expires overdue reviews. At start-up the worker first runs `recover()`, which finishes any approval that was saved but not completed.

Saved item	Where	When written
Claim state (inputs, step results, errors, retries, human actions, token metrics)	states/<claim>.json (atomic replace)	Before every step and at the end
Pending review, decision, expiry, content version	SQLite `reviews`	At S10; on every decision or timer
Every incoming event and its outcome	SQLite `events` (event_id is the key)	On arrival; outcome set when done
Payments and customer notifications actually recorded	SQLite `payments`, `notifications` (unique keys)	At the moment of the external effect
One line per step and event	logs/run_log.jsonl	Every step

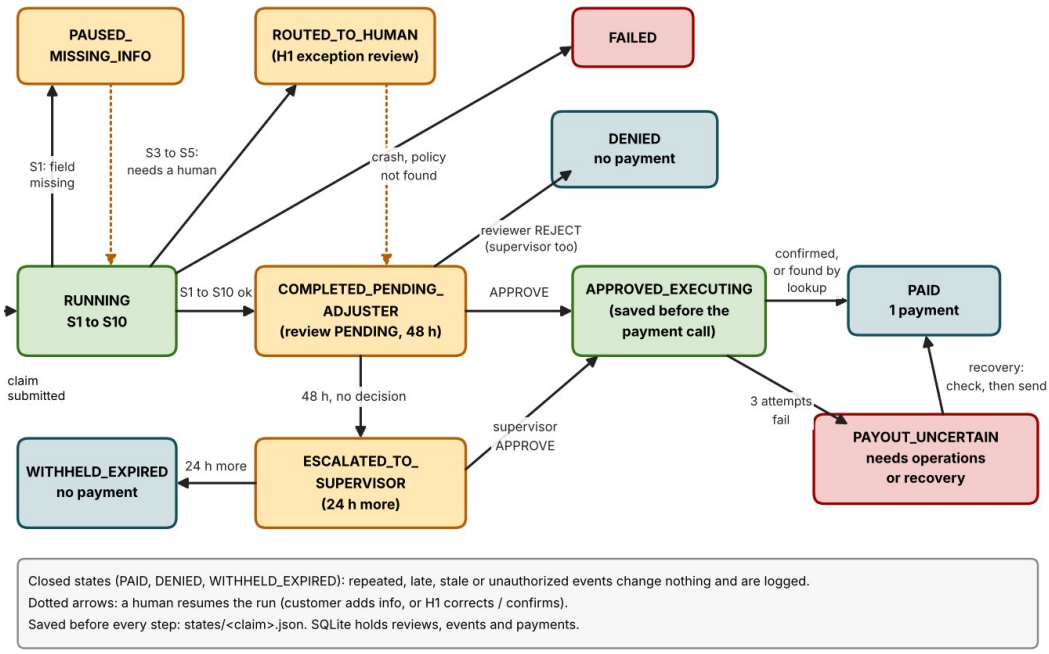


Figure 2: Claim state diagram: normal progress, waiting, failure, escalation and the closed states.

Duplicate, Late and Unexpected Events

Situation	How it is detected	Result (tested)
Same event delivered twice	event_id already in `events` with an outcome	DUPLICATE_IGNORED
Second decision with a new event_id; conflicting decision after approval	Review no longer open; compare-and-set update lets only one decision win	LATE_IGNORED(APPROVED)
Decision after expiry	Timers are applied before the decision is read	LATE_IGNORED(EXPIRED)
Claim changed since the review was shown	Content hash differs, or review already SUPERSEDED	STALE_VERSION / LATE_IGNORED(SUPERSEDED)
Wrong person; unknown review; unknown event type; no event_id	Role check; lookup; type check	NOT_AUTHORIZED, UNKNOWN_REVIEW, UNEXPECTED_EVENT, INVALID_EVENT
Approval of a fallback-scored claim without acknowledgement	Review flag fallback_score	NEEDS_ACK_FALLBACK (review stays open)

Situation	How it is detected	Result (tested)
Worker dies mid-event	Event row has no outcome; claim saved as APPROVED_EXECUTING	Event reprocessed; recover() finishes the payment

Safe retries and a lost confirmation. The payment is sent with the key `payout:<review_id>`. Before the first send, and again after any lost confirmation, the workflow asks the payment system whether that key already exists. If it does, the claim is marked PAID and nothing is sent again; if the service is down, the send is retried up to 3 times (1 s, 2 s), then the claim is marked PAYOUT_UNCERTAIN and `recover()` retries later, again checking first. Our simulated receiver also ignores a repeated key, as real payment APIs do; if a real one did not, the lookup is the only protection.

Milestone 2 feedback fixed. A reviewer's correction is validated again (schema and cross-check with the form) before it is used; an invalid correction is refused and the claim is unchanged. A valid correction creates a new review and the old approval can no longer pay (test 9). `resume_claim()` now ignores claims in closed states and resets retry counters while keeping the old counts in `retry_history` (tests 4b, 10).

4. Test Results

All tests ran on a team member's Mac with the real gpt-oss:20b model. Tests 1 and 8 run the whole workflow including both LLM calls. To avoid repeating 45-second model calls, tests 2–7 and 9–11 copy the finished claim from test 1 under a new ID and give it its own review; they test the review, event and payment layers, not the model. After each case we counted rows in the simulated receiver (`payments`, `notifications`) rather than trusting the claim status.

Case and input	Expected	Actual and evidence	Interpretation
1 Approved run. TC1 narrative through S1–S10; adjuster_1 approves	PAID; 1 payment equal to the proposed amount	APPROVED_PAID; ledger: 1 payment of $50,590.00 = proposed; 1 notification	End-to-end path works; money moves only after approval
2 Rejection. adjuster_1 rejects	DENIED; 0 payments; 1 notification	REJECTED; claim DENIED; payments 0, notifications 1	Rejected action withheld
3 Timeout. No decision for 49 h, then 73 h; adjuster_1 approves at 50 h; supervisor_1 approves at 74 h	ESCALATED, then WITHHELD_EXPIRED; late approvals refused	ESCALATED_TO_SUPERVISOR → WITHHELD_EXPIRED; NOT_AUTHORIZED; LATE_IGNORED(EXPIRED); payments 0	Silence never pays; an expired review cannot be revived
4 Duplicate event. Approve delivered twice, approved again with a new id, then rejected	1 payment only	APPROVED_PAID; DUPLICATE_IGNORED; LATE_IGNORED(APPROVED) twice; payments 1, notifications 1. Late resume_claim on the PAID claim ignored	Repeats cause no second effect
5 Interruption. Worker dies after the approval is saved, before the payment call; restart; broker redelivers the event	APPROVED_EXECUTING with 0 payments, then PAID with 1	As expected; redelivered event LATE_IGNORED(APPROVED); payments 1	Recovery from saved state; no double payment
6 Uncertain outcome. Payment recorded but the response is lost	Check receiver; PAID; no resend	Status text: recorded, not resent; payments 1	Verified what happened before acting
7 Service down. 3 failed attempts, then service returns	PAYOUT_UNCERTAIN (0 payments), then PAID (1)	As expected; payments 1	Bounded retries; later recovery
8 Fallback score. Risk model forced to fail (real LLM run)	Band UNVERIFIED, not LOW; approval without acknowledgement refused	Score 0, band UNVERIFIED; NEEDS_ACK_FALLBACK, payments 0; with acknowledgement PAID	Fixes the M2 weakness (score 0 shown as LOW)

Case and input	Expected	Actual and evidence	Interpretation
9 Correction. Amount +50%, then a valid vehicle correction; approve old and new review	Bad correction refused; old review cannot pay; new pays once	Refused (narrative $77,385 vs form $51,590); old LATE_IGNORED(SUPERSEDED); new APPROVED_PAID; payments 1	Approval applies to the corrected version
10 Counters. Claim stopped with retries {S3_reprompt: 2}; reviewer resumes	Counters cleared, history kept	retries {}, history [{S3_reprompt: 2}], new review pending	Fixes M2 limitation
11 Unexpected. Unknown review, wrong type, clerk_9, no event_id	Four distinct refusals; 0 payments	UNKNOWN_REVIEW, UNEXPECTED_EVENT, NOT_AUTHORIZED, INVALID_EVENT; payments 0	Bad input cannot cause an effect

Totals. 13/13 Milestone 3 checks passed; 7 payments are recorded for the 7 claims that were approved and paid (`results/m3_payments.csv`). The 19 Milestone 2 scenarios still pass on the new code (8 of 12 claims completed, 3 routed to H1, 1 paused; 32.9 s and 2 LLM calls per completed claim).

Failures and Weaknesses Found

- **Misleading fallback explanation.** For the fallback claim the LLM wrote that the score of 0 "indicates no automated red flags". The reviewer warning and the UNVERIFIED band protect the reviewer; the explanation is not trusted.
- **Ungrounded wording.** In the TC1 explanation the model refers to "the available police report", which it never saw; it only had the narrative and summary. Reviewers see the evidence beside the explanation.
- **Redelivery outcome.** After a crash the redelivered event is reported LATE_IGNORED(APPROVED), not DUPLICATE_IGNORED, because the first attempt never finished. The effect is still correct; we kept the behaviour and documented it.
- **Process failure.** One script (`review_cli.py`) was saved empty on one machine and still passed a syntax check; we found it only by running `review_cli.py list`. The README run order now exercises the commands.
- **Not tested:** two workers at the same time, a real payment API, and real reviewers. The first model call after loading took 61–76 s.

5. Production Operation and Cost

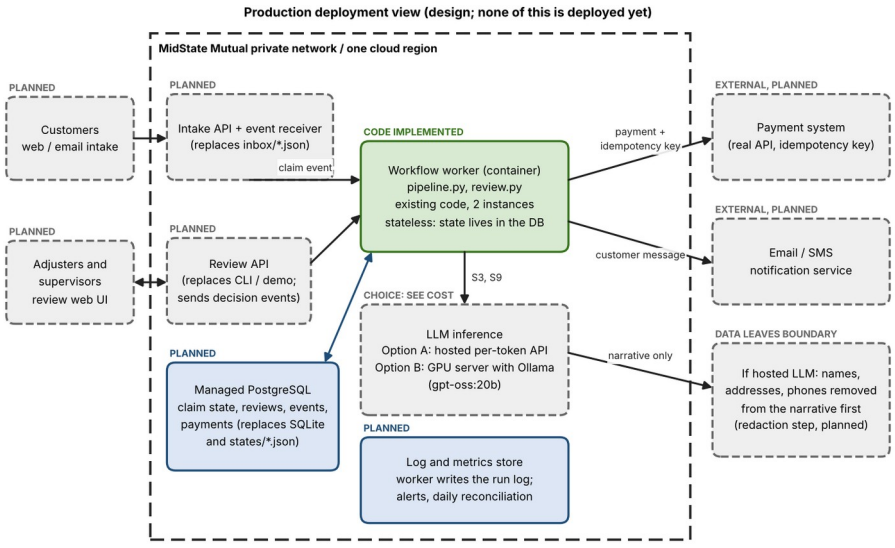


Figure 3: Deployment view. The worker code is the implemented prototype; everything else is design. State and logs reside in PostgreSQL and the log store, not on the worker.

Where things run. Two stateless worker containers run `pipeline.py` and `review.py` inside the company network; claim state, reviews, events and payments move from SQLite and JSON files to a managed PostgreSQL database; the run log and metrics go to a log store; the payment system and notification service are external. LLM inference is either a hosted per-token API (narrative redacted first, because it leaves the boundary) or a GPU server running Ollama (Option B).

Cost per Completed Claim and per Month

Measured over 12 completed claims on the real model: 2 LLM calls, **634 input and 553 output tokens**, 33.2 s of model time per claim. **Rates (October 2026):** hosted gpt-oss-20b $0.05 per 1M input and $0.20 per 1M output tokens (third-party price page citing OpenAI); AWS g5.xlarge GPU server $1.006 per hour. Token cost per claim = 634 × $0.05/1M + 553 × $0.20/1M = **$0.00014**.

Option (3,000 claims/month assumed)	Model cost per month	Machine total (+ $55 app, database, logs)	Machine cost per claim	All-in per claim (+$4.50 review)	All-in per month
A. Hosted per-token API	$0.43	$55.43	$0.0185	$4.52	$13,555
B. GPU server, business hours (176 h)	$177.06	$232.06	$0.0774	$4.58	$13,732
C. GPU server, 24x7 (730 h)	$734.38	$789.38	$0.2631	$4.76	$14,289

Assumptions we made (not measured): 3,000 claims per month; 6 minutes of adjuster time per reviewed claim at $45 per hour ($4.50 per claim, $13,500 per month); $30 per month for application hosting and $25 for database and logs. Development, support and electricity for the prototype laptop are not counted. A 24x7 GPU beats per-token pricing only above about 5.2 million claims per month.

Strongest cost-saving opportunity. Review time, not the model: the model is at most $734 of about $14,000 per month. In our 12 completed test claims 8 (67%) scored LOW. If LOW claims were paid under monitoring with a 10% audit sample instead of approval before execution, review cost would fall from $13,500 to about $5,400 per month (−$8,100, −60%). **Trade-off:** money leaves before a person looks. The ML model has recall 0.735, so about one fraudulent claim in four scores lower than it should, and a single wrongly paid claim of the size of TC1 ($50,590) costs about six months of the saving. The sample is small and synthetic, so we do not adopt this in Milestone 3; Milestone 4's risk assessment must weigh it. Switching from a GPU server to the hosted API saves $177–$734 per month but sends narratives to a third party.

Latency, Reliability and Operational Records

Topic	Design
Completion time users need	Review package ready within 2 minutes of submission (assumed: adjusters work in hours, not seconds). Measured 17–53 s per claim, average 32.9 s. Reviewer decision within 48 h, supervisor 24 h more
Retry and timeout limits	LLM call: 3 attempts (1 s, 2 s backoff); extraction: 3 attempts; payment: 3 attempts (1 s, 2 s), then PAYOUT_UNCERTAIN. Review deadlines are implemented. A per-call network timeout for the LLM (120 s) is planned, not implemented
Service unavailable	LLM down: claim routed to H1 after 3 attempts (tested, 3 s). Risk model down: fallback score, UNVERIFIED. Payment down: PAYOUT_UNCERTAIN, recovered later (tested). Database down: worker stops; event files stay in the inbox and are replayed (queue planned)
Records to reconstruct a run	run_id, claim_id, step, status, attempt, latency (run log); review_id, content version, reviewer, decision, times (reviews); event_id and outcome (events); payment key, amount, time (payments, notifications); tokens and errors (claim state)
Problems detected (planned daily reconciliation and alerts)	Payments that do not match approved reviews; any PAYOUT_UNCERTAIN; escalated or expiring reviews; share of fallback scores; LLM failure rate; duplicate-event rate

6. Contributions, AI-Use Disclosure and Remaining Limitation

Member	Milestone 3 contribution	Evidence location
Ishita Varma Kalindindi	Human-review design: reviewer screen, approval scope (action, target, content version, expiry), outcomes for approve, reject and timeout; wrote Sections 1-2 of this report.	src/review.py (create_review, render_review, content_hash); review_cli.py
Raveena Pavuluri	Saved state and event handling: SQLite store, atomic state files, event inbox and worker, duplicate/late/unexpected handling; owns the repository and access.	src/store.py, src/state.py; review.py (handle_event, sweep); inbox/ events
Nikhithareddy Obili	Mac and Ollama setup; ran the real-model tests; cost model with measured tokens and the production cost estimate (Section 5).	run_m3_tests.py, cost_model.py; results/m3_*.csv, results/cost_model.csv
Tejaswini Waghmare	Safe retries and failure simulation: payment/notification services with fault injection, lost-confirmation and uncertain-outcome tests; diagrams; README.	src/external.py; review.py (execute_payout, recover); README.md; docs/ figures
Kiarash Rezaei	Milestone 2 feedback fixes: resume_claim status guard, retry-counter reset, correction re-validation; fallback-score UNVERIFIED band; M2 regression tests.	src/pipeline.py; run_tests.py; run_demo.py

AI-use disclosure. Claude (Anthropic) was used to draft the Milestone 3 code (store, external, review, tests, cost model), the diagrams and the report text. No other AI tool was used. All scripts were run by the team on a Mac with the local model; every result in this report comes from the console output and `results/*.csv`. Prices come from public pages read in October 2026 and were not independently verified. The team reviewed the code and the numbers.

Most important remaining limitation. The payment and notification services are our own simulations. The protection against double payment (keys, lookup before retry) was tested against that simulation only; a real payment API's behaviour, two workers running at once, and real reviewer behaviour are untested. Governance, security and the risk assessment, including the monitoring-with-audit idea above, belong to Milestone 4.

References

1. A. Aqqad, "insurance_claims," Mendeley Data, Version 2, 2023. DOI: 10.17632/992mh7dk9y.2. License: CC BY 4.0.

2. Hosted gpt-oss-20b token prices: anotherwrapper.com/llm-pricing/gpt-oss-20b (accessed Oct. 25, 2026). AWS g5.xlarge on-demand price: doit.com/compute/spot/us-east-1/g5.xlarge (accessed Oct. 25, 2026).

3. Ollama, model serving platform, ollama.com.